

Assignment 3 - Report 1

324827328, 213713522

May 2025

Case 1

In the following case, we took the original language from question 1, but we made it "harder" by making the letters sequences only equal to 1, and the numbers sequences length in the range of between 100-150. An example of how the a positive and negative case would look like:

Positive Example

"3741834944194224948285419448483269944349648836933825723729717693929234199111737213a7287881959253476427673635445121554322469493222632676789154271279962195382224815538667973567514286135858616933266871414256b7614511475147249177663198819279682979c2769841487271461625116347457577975d46295347168275161935991444676773618149917218688311919131558736446751275121".

As you can see, "b" comes after "c"

Negative Example

"156457516399155457247435855512275664789889424374749826195549766219327347556183a27614731198349315129234679959612638258c86187863382912312733555736639547379458158637237599146563418945745125872b729654148367245938738575479645266498746667535721279781293993234764322577116566332321159655934779887563186177676334598d4973582942775313845858421478312277385564781269796984992584368813978434248364418755865432647961365286968469514175234423383566684914446324".

In the negative example, "c" comes before "b".

Why would this languages be hard to distinguish?

1. Dominance of Random Digits: Each sequence contains five blocks of digits, each of which can be up to 150 characters long. These digit blocks dominate the overall sequence length (up to 750 out of 755 characters), making

the letter-based structure a very small part of the input. This dilutes the signal carried by the crucial letters (a, b, c, d), making it harder for the LSTM to detect them amidst the noise.

2. Minimal Alphabetic Difference: The only structural difference between the positive and negative classes is the order of two specific characters:

- Positive: a $\rightarrow$ b $\rightarrow$ c $\rightarrow$ d
- Negative: a $\rightarrow$ c $\rightarrow$ b $\rightarrow$ d

This small permutation in the middle of the sequence is difficult for an LSTM to detect, especially when surrounded by large, random numeric sequences.

3. Long-Term Dependency Requirement: The LSTM must retain information about the specific ordering of characters (b and c) across long stretches of irrelevant digits. Standard LSTMs struggle with such long-term dependencies, particularly when the important signal is sparse and the irrelevant content (random digits) is dominant.

4. Vanishing Gradient Effects: As the length of the sequence increases, the gradients associated with early characters (a, b, etc.) diminish during back-propagation through time. This makes it harder for the LSTM to learn meaningful signal from early tokens, which is crucial since the only class-differentiating signal resides in the order of a few early-placed letters.

Did we manage to fail the LSTM acceptor?

We trained our model for 100 epochs, which is 20 times more then the amount we had to train for question 1. We used the same learning rate of 0.001, this is a very small learning rate, but we are mainly focusing on understanding if the can learn to separate the sequences, so its fine if we wont get optimal accuracy. Lets look at the results:

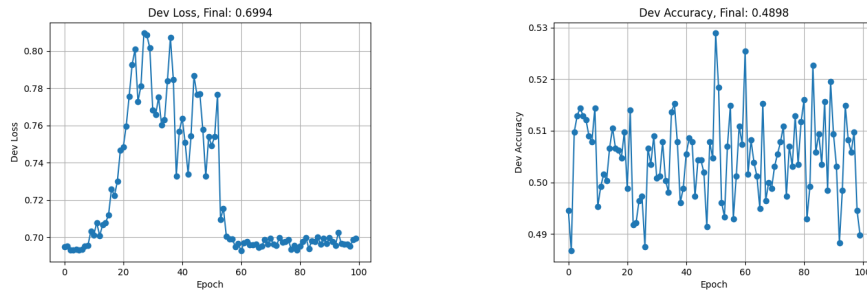


Figure 1: Dev Set Loss And Accuracy

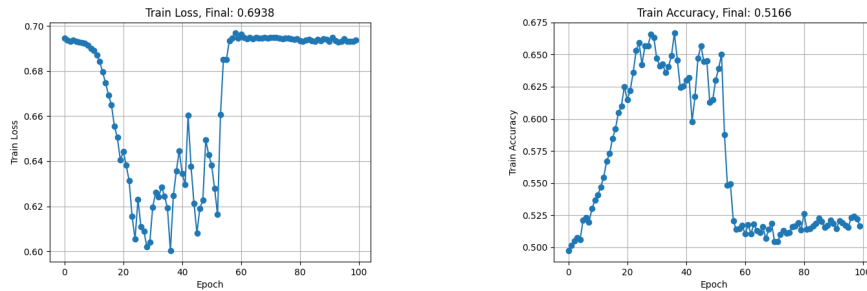


Figure 2: Train Set Loss And Accuracy

Its clear the model didn't generalize well on the dev-set, with the accuracy always being around 0.5. This means the model was mainly guessing the samples result. The model did manage to improve the train accuracy for some time, to around 0.66, but then went down. This indicates that the model might have overfitted the train-set. Both the train and dev loss stayed quite high over all training.

Case 2

In the following case, we used a weakness of LSTMs learned in class. LSTMs cant count! So we made a language that for a sequence to be a part of, the sequence must have 4 sequences of "b" appearing 15 times. We also added numbers in between the 'b' sequences of lengths in the range of 10 to 20. For example:

Positive Example

53376587331214239bbbbbbbbbbbbbb2718988557551655bbbbbbbbbbbbbb25
38936577bbbbbbbbbbbbbb76612767655bbbbbbbbbbbbbb7254831717245816
3.

In the above example, we can see between each sequences of numbers there are 15 'b's.

Negative Example

8956288958894bbbbbbbbbb312589738324bbbbbbbbbbbbbb78981248257
189289427bbbbbbbbbbbbbb32449188171567bbbbbbbbbbbbbb5992511534.

In this example, there are sequences between the numbers that dont contain 15 "b"s.

Why would this languages be hard to distinguish?

1. Quantitative Rather Than Qualitative Difference: Unlike the previous example, the distinction between positive and negative sequences does not lie in the order of characters, but in the total quantity of a specific character (namely, the letter **b**). Positive sequences contain exactly four blocks of **b**'s, each of a fixed length, while negative sequences contain the same number of blocks, but with varying lengths such that the total number of **b**'s differs from the positive case.

2. Long Sequences with High Noise: Each sequence is composed of 5 digit blocks and 4 letter blocks. Each digit block can be up to 20 characters long, adding noise and variability. This pushes the total sequence length to around 180 characters, with only around 40–80 characters (depending on randomness) carrying class-relevant signal.

3. Global Counting Required: To correctly classify the sentence, the model must essentially count the number of **b**'s across the entire sequence and compare it to a target quantity. LSTMs are known to struggle with precise counting tasks, especially when the tokens to be counted are interspersed with noise (digits) and the total count is the only clue for classification.

Did we manage to fail the LSTM acceptor?

We trained this model the same as the in Case 1. Lets look at the results:

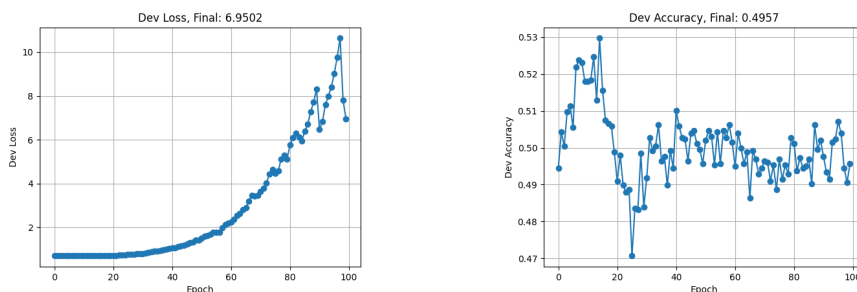


Figure 3: Dev Set Loss And Accuracy

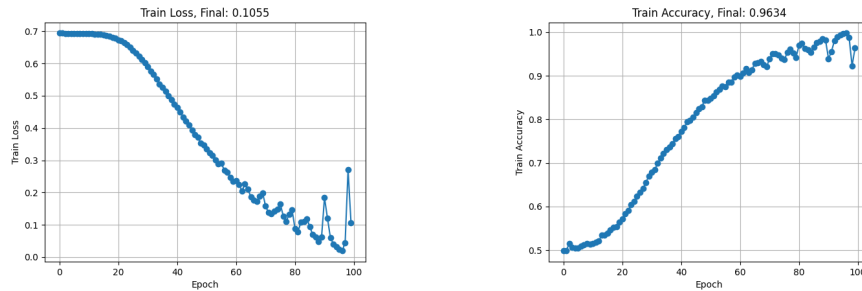


Figure 4: Train Set Loss And Accuracy

Here, we saw interesting results. The model did manage to learn, but it only manages to overfit the training data. We can clearly see the dev loss is very high, and the dev accuracy is around 0.5, meaning the model is "guessing" the results of the dev set examples.

Case 3

In the following case, we use another weakness of LSTMs learned in class. We learned that LSTMs update their entire memory each time, making it very hard for them to remember things that appeared a long time ago. So, in the next case, for a sequence to be positive, it must start with the letter "b", otherwise it's negative. Let's look at some examples:

Positive

b1597253491645439325951891997218915261731482253218312722528494511696412173731697483428172344935699626627635546428876442841744.

We can see the sequence's first char is "b"

Negative

3544285752918552869415758852251744784991853586878764342212595461698848672192985981916941564928172265621157317716384713986354499821931555644916853.

Here, we can see the sequence is just some random numbers.

Why would this language be hard to distinguish?

1. Minimal Signal: The only distinguishing feature between the positive and negative classes is the presence of a single character 'b' at the **very beginning** of the sequence:

Positive: b + digits Negative: digits

This means the classification depends entirely on detecting whether the first character is 'b'.

2. Extreme Class Imbalance in Features: The rest of the sequence—composed of 100 to 150 digits—is identical in structure and distribution for both classes. As a result, over 99% of the tokens are statistically indistinguishable across classes. The meaningful information occupies only the first character (less than 1% of the sequence), making the learning signal extremely sparse.

3. Vanishing Gradient and Forgetting: Although the distinguishing 'b' appears at the start, LSTMs are prone to *forgetting* information as it flows through the network due to vanishing gradients over long sequences. Since the rest of the sequence can be up to 149 digits long, the model may lose or distort the memory of the initial character by the time it reaches the output.

Did we manage to fail the LSTM acceptor?

We trained this model the same as the in Case 1. Lets look at the results:

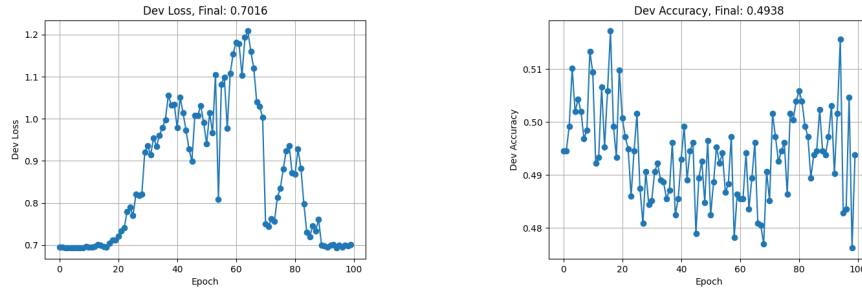


Figure 5: Dev Set Loss And Accuracy

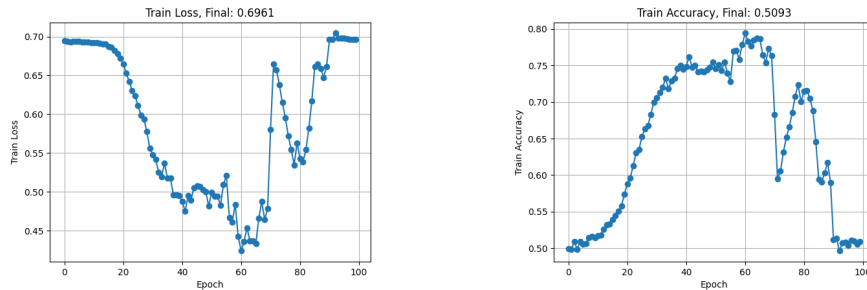


Figure 6: Train Set Loss And Accuracy

Here, we can see the model failed again. The training loss and accuracy did improve for some time, while not improving the dev loss and accuracy, suggesting overfitting. Furthermore, if we look at the dev accuracy, we can see it

stayed around 0.5 for the entire training, meaning- the model "guessed" the dev examples results.